
VISUAL–TEMPORAL RETRIEVAL–AUGMENTED GENERATION FOR LECTURE QUESTION ANSWERING

Meenakshi Mittal, Micah Mok, Anthony Kuang

Department of Electrical Engineering and Computer Sciences

University of California, Berkeley

Berkeley, CA 94720, USA

{meenakshi.mittal, micahmok, anthonykuang}@berkeley.edu

ABSTRACT

Lecture videos are deeply multimodal. An answer to a single student question may live in spoken explanation, on-screen text, slide images, tables, charts, or transient gestures. Transcript-only and slide-text-only retrieval pipelines collapse this richness into a single channel and consequently fail on questions that rely on visual interpretation or precise temporal grounding. We introduce a manually annotated benchmark of 156 question–answer pairs across three UC Berkeley courses (cognitive neuroscience, advanced NLP, and data science), spanning 7 example categories: text, images, tables, charts, complex layouts, motion-dependent content, and audio. We then propose a visual–temporal retrieval-augmented generation (RAG) pipeline that aligns OCR text, ASR transcripts, and vision–language model frame descriptions inside fixed-length time windows with a visual carry-forward rule, embeds each multimodal chunk into a single dense vector, and retrieves it at inference time conditioned on either the full corpus or a lecture-filtered subset. On our benchmark, lecture-filtered retrieval lifts LLM-judged accuracy from 20.4% to 82.0% on DATA 100, from 32.7% to 58.8% on COGSCI C127, and from 63.0% to 83.3% on CS 288, while keeping per-question latency on the order of three seconds and outperforming direct full-video conditioning of frontier multimodal models in our evaluated setting. The dataset, pipeline, and evaluation harness are released to support reproducible work on lecture QA.

1 INTRODUCTION

Educational video is a uniquely multimodal medium. A single ten-second segment of a lecture may contain a slide title in OCR-able text, a spoken example, a hand-drawn diagram, a chart with quantitative axes, and an instructor pointing at a specific region of the screen. Students seeking to revisit such a moment do not naturally distinguish between modalities; they ask questions like “*what was the y-axis label on the scree plot?*” or “*which archery target shows marks that are clustered together and off-center?*”, and they expect answers grounded in the actual lecture rather than a generic textbook.

Existing approaches to lecture question answering largely operate on a single channel. Transcript-only pipelines retrieve over automatic speech recognition (ASR) output, which fails whenever the answer is written on a slide and never spoken aloud. Slide-text-only pipelines run optical character recognition (OCR) over slide exports, which loses the temporal context that distinguishes one mention of a term from another and which silently drops dynamic content such as videos shown during class. Direct video-conditioned generation with frontier multimodal models is technically possible but, as we show in Section 5, surprisingly brittle: lecture-length context windows induce both latency blowups and accuracy regressions on questions that require precise local evidence.

We argue that lecture QA should instead be cast as a visual–temporal retrieval problem. Concretely, we (i) align multiple modalities by timestamp into a single chunk representation that preserves cross-modal evidence, (ii) embed those chunks for vector retrieval, and (iii) pass the retrieved chunks

together with their associated slide images to a vision–language model (VLM) at generation time. To test this hypothesis we construct a 156-question benchmark across three real Berkeley courses with deliberately difficult, course-specific questions, and we evaluate three baselines along the spectrum from zero-shot generation to full lecture-video conditioning, plus two variants of our proposed pipeline that differ in retrieval scope.

The contributions of this work are as follows. First, we introduce a manually annotated visual–temporal lecture QA benchmark covering 7 question categories drawn from three courses. Second, we present a time-aligned multimodal chunking pipeline whose retrieved units carry OCR, ASR, and VLM-derived visual descriptions in a single embedding, with a visual carry-forward rule that preserves slide context across window boundaries. Third, we report an empirical comparison across no-context, video-conditioned, and RAG conditions, showing that lecture-filtered multimodal retrieval improves LLM-judged accuracy significantly over the no-context baseline while keeping per-question latency under three seconds. Finally, we conduct a fine-grained error analysis that identifies fine-detail chart reading and motion-dependent reasoning as the remaining bottlenecks and motivates a frame-image pass at the final generation step.

2 RELATED WORK

Question answering over lecture and educational video. Recent work has begun to study question answering directly over lecture and educational video. EduVidQA introduces a benchmark for answering long-form student questions from lecture videos with multimodal LLMs (Ray et al., 2025). Wadhwa et al. (2025) similarly study multimodal QA in educational settings and motivate course-grounded question answering that reaches beyond transcript-only access. Earlier systems such as ClassTranscribe demonstrate the value of searchable lecture transcripts and automated captioning for navigating instructional content (Mahipal et al., 2019). Together, these works establish lecture QA as a meaningful educational task while also suggesting that transcripts alone are insufficient for many natural student questions.

Multimodal lecture representations. A second relevant line of work emphasizes that lecture materials are inherently multimodal. The Multimodal Lecture Presentations Dataset highlights that educational slides combine text, figures, and spoken context in ways that require more than simple OCR or transcript matching (Lee et al., 2022). Wang et al. (2024) show that visual slide context can improve transcription quality in technical videos through knowledge-intensive multimodal ASR. HowToCaption demonstrates that LLMs can transform noisy instructional-video narrations into cleaner, better aligned descriptions (Shvetsova et al., 2023). These results motivate representing lecture content with aligned audio, OCR, and visual descriptions rather than treating each modality in isolation, and inform our design choice to bundle all three streams inside each retrieval unit.

Long-video understanding and retrieval. Recent work on long-video understanding motivates retrieval-based approaches for our setting. VideoRAG argues that retrieval is necessary for very long videos, where full-context prompting is both expensive and inaccurate (Ren et al., 2025). Video-MMLU shows that lecture understanding remains challenging even for strong multimodal models across many disciplines (Song et al., 2025). LEMON further highlights the difficulty of temporal multimodal understanding in instructional videos when answers depend on events or dependencies unfolding over time (Yu et al., 2026). Taken together, these works motivate a benchmark and baseline suite that explicitly tests when transcript-only retrieval is sufficient and when richer multimodal grounding over slides, visuals, and time is required.

3 DATASET

Source material. We construct a new manually annotated evaluation dataset for visual–temporal RAG over lecture videos. The source material is drawn from three undergraduate courses at UC Berkeley: COGSCI/PSYCH C127 (Cognitive Neuroscience), COMPSCI 288 (Advanced Natural Language Processing), and DATA 100 (Principles and Techniques of Data Science). The three courses were chosen to span instructional styles: cognitive neuroscience features long stretches of speech and frequent reference to in-frame motion, NLP relies heavily on equation-laden slides, and data science combines code demonstrations with charts and tables.

Table 1: Dataset schema. Each record is a single question–answer pair with an associated lecture, time window, and optional visual context.

Field	Description
lecture_id	Lecture recording filename
ts_start, ts_end	Timestamp window of the answer segment
question	Student question
question_image	Optional image attached to the student question
answer	Ground-truth answer
answer_image	Optional image attached to the ground-truth answer

Collection process. We download lecture recordings and identify moments where the instructor poses a question, explains a key concept, or presents course-specific information. For each such moment, an annotator records a natural-language question, a concise ground-truth answer, the lecture of origin, the start and end timestamps of the relevant segment, and images of the relevant slides. Questions are deliberately constructed to be difficult to answer with baseline LLM capabilities, requiring attentive and specific understanding of lecture content rather than recall of widely available facts.

Schema. Each record contains the fields listed in Table 1: a lecture identifier, a timestamp window, the question text, an optional question image, the ground-truth answer, and an optional answer image. The image fields support questions that either reference a visual region (“*what is depicted in green in the bottom right?*”) or whose answer is itself most naturally a visual artifact.

Statistics. The dataset contains 156 question–answer pairs spanning 60 lectures across the three courses, split as 52 questions on COGSCI C127, 50 on DATA 100, and 54 on CS 288. Segment durations range from a single timestamp to roughly three minutes, with a median of approximately 30 seconds. Questions fall into seven informal categories: (1) questions answerable by reading on-screen text alone, (2) questions requiring interpretation of images, (3) questions involving tables, (4) questions involving charts or graphs, (5) questions requiring understanding of complex slide layouts, (6) questions answerable by listening to instructor audio, and (7) dynamic motion-dependent questions such as describing a patient’s gait from video.

Examples. Representative questions illustrate the range of evidence required:

- *Table.* DATA 100, Lecture 4, 56:18–57:39. “What was Stephen A. Douglas’ popular vote count, according to the elections dataframe?” Answer: 1,380,202.
- *Image.* DATA 100, Lecture 18, 30:03–33:03. “Which archery target shows marks that are clustered together and off-center?” Answer: D.
- *Speech.* DATA 100, Lecture 1, 33:40–33:48. “Who is the data scholars lead?” Answer: Abby.

4 METHOD

We treat lecture QA as visual–temporal retrieval-augmented generation. A lecture is decomposed into time-aligned multimodal chunks, each chunk is embedded into a single dense vector, and at inference time the top- k chunks together with their associated slide images are passed to a VLM that produces the final answer. We additionally describe an output-generation stage that turns retrieved evidence into a narrated video. The pipeline has five stages.

Stage 1: Per-modality extraction. For every lecture we extract three streams that are each natively timestamped using Azure-hosted models: OCR text from sampled video frames using Azure AI Vision’s Read OCR engine, ASR transcripts from the audio track using the Azure OpenAI Whisper model, and VLM-generated frame descriptions using our Azure OpenAI gpt-5 . 4 deployment. The

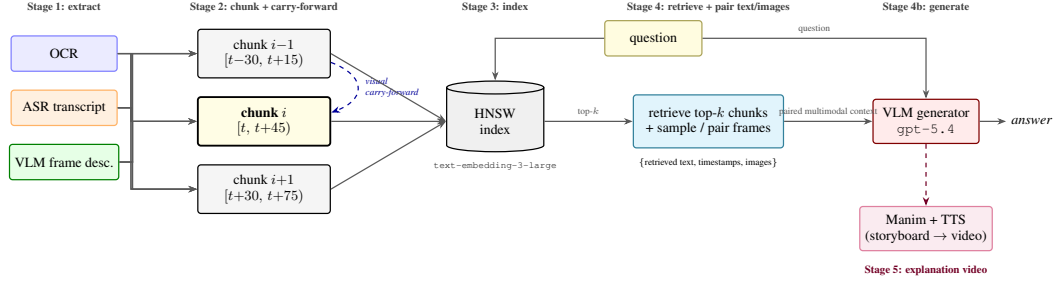


Figure 1: Pipeline overview. OCR, ASR, and VLM frame descriptions are aligned by timestamp into overlapping multimodal chunks ($w=45$ s, $o=15$ s); the dashed arrow shows the visual carry-forward rule, which injects the latest pre-window OCR and frame-description rows into chunk i so that a slide appearing just before t_0 still influences retrieval. Chunks are embedded with `text-embedding-3-large`, indexed in HNSW, and retrieved at query time; the top- k chunks plus the question are passed to a VLM generator (`gpt-5.4`) to produce the answer.

OCR stream captures on-screen text, the ASR stream captures spoken explanation, and the VLM stream captures non-textual visual content such as charts, diagrams, and motion. Frames are sampled every two seconds; we use image hashing (specifically pHash) and a manually determined threshold to detect whether the sampled frame has changed materially since the previous sample, and the OCR and fresh frame description are regenerated only on detected change. This periodic motion-detection step removes the dominant source of redundancy in slide-driven lectures, where most consecutive frames are visually identical.

Stage 2: Time-aligned multimodal chunking with visual carry-forward. The central design choice of our method is to bundle all three modalities into a single retrieval unit indexed by time. We tile each lecture with overlapping windows of length w seconds advancing by step $w - o$, where o is the overlap length. For each window $[t_0, t_0 + w)$ we collect the ASR text whose token timestamps fall inside the window, the OCR rows whose timestamps fall inside the window and directly before it, after thresholding lines under $\ell_{\min}$ characters to suppress menu chrome, and the VLM frame descriptions whose timestamps fall inside the window and directly before it. To prevent dense visual streams from dominating the chunk text, OCR and frame rows inside the window are subsampled to a minimum gap of g seconds.

A naive in-window scheme misses a common slide-driven failure mode: a slide that appears just before the window start and is still on screen during the window contributes no OCR or frame row to the chunk, even though the speaker is referring to it. We address this with a *visual carry-forward* rule: for each window we additionally inject the latest OCR row and the latest frame-description row whose timestamp falls strictly before t_0 , deduplicated against the in-window content. The chunk text is then the concatenation of three modality-tagged blocks (`[Speech]`, `[OCR / on-screen]`, `[Frame description]`), each of which is omitted if empty.

In our reported configuration we use $w = 45$ s, $o = 15$ s, $\ell_{\min} = 12$, and $g = 10$ s. Each chunk additionally retains a list of which modalities contributed to it and which were carried forward, which we use as a diagnostic.

Stage 3: Dense indexing. Each chunk’s text payload is embedded by Azure OpenAI `text-embedding-3-large` and uploaded to an Azure AI Search index whose schema includes `course_id`, `lecture_id`, `lecture_number`, `chunk_index`, `t_start`, `t_end`, the modality-tagged text, the source-modality tags, and a single dense `content_vector` field served by an HNSW vector profile. `lecture_number` is parsed once at index time from the lecture filename so that downstream filtering does not require string matching. We additionally support a local backend in which the same chunk format is embedded with BGE-small and served from an cosine-similarity index; both backends are tested by the same evaluation harness and produce comparable retrieval quality, so we report all main results with the Azure backend.

Stage 4: Retrieval and generation. At query time the question is embedded with the same encoder used at index time, and the top- k chunks are returned by HNSW vector search. We compare two retrieval scopes that differ only in their server-side filter. *Global retrieval* sets a single `course_id` filter and searches across every chunk in the course, modeling the realistic deployment where a student does not pre-specify which lecture their question came from. *Lecture-filtered retrieval* additionally constrains `lecture_number` to the lecture associated with the question, modeling the deployment where a student is reviewing a specific lecture and asks an in-context question. We do not perform any reranking beyond the initial dense retrieval; the temporal-hit rate defined in Section 5 measures whether any retrieved chunk overlaps the annotator-marked answer segment, and serves as our primary diagnostic for retrieval quality independent of generation.

The retrieved chunks are formatted into a single context string, with each passage prefixed by its `lecture_id` and time interval so that the generator can attribute claims temporally. The generator is prompted with a brief teaching-assistant system prompt that instructs it to answer like an exam answer key (the shortest correct phrase, name, label, number, or sentence) and to say so when it does not know; the full prompt is reproduced in Appendix A.1.

Stage 5: Output video generation. Beyond returning a text answer, we introduce a downstream stage that converts retrieved multimodal lecture evidence into a narrated explanation video.

The top- k retrieved chunks and their associated slide images are passed to a vision-language model (via OpenRouter, defaulting to Gemini) with a structured prompt that instructs it to decompose the answer into a *storyboard JSON*: an ordered list of typed scene objects drawn from six pedagogical templates. DEFINITION renders a term with its definition and optional example. THEOREM presents a formal statement in a framed box. PROOF enumerates proof steps with $\LaTeX$ -rendered mathematics. EXAMPLE walks through a worked example with sub-steps revealed in turn. EXERCISE presents a problem followed by a revealed solution. FIGURE displays a retrieved slide image with a caption, suitable for chart, diagram, or table answers. Each template carries the content fields appropriate to its role (e.g., `term` and `definition` for DEFINITION; `image_ref` and `caption` for FIGURE), plus a `voiceover` field present in every type.

The prompt instructs the VLM to select scene types appropriate to the question: a factual lookup typically produces a single DEFINITION or FIGURE scene, while a conceptual or derivation question may produce a four-to-six-scene sequence. The VLM is further instructed to ground each scene’s content in the retrieved text and to copy voiceover phrasing from the ASR transcript where the instructor’s own words are available.

The storyboard JSON is rendered into a narrated video using Manim Community Edition and the ElevenLabs TTS API. A code-generation step maps each scene’s `type` and content fields to a typed `VoiceoverScene` subclass that handles both the visual layout (Manim animation calls) and narration (the `voiceover` string). ElevenLabs is registered as the TTS provider within Manim’s `VoiceoverTracker`, which synthesizes audio and aligns animation timing to speech duration automatically. Scenes are rendered in parallel as isolated Manim subprocess calls and concatenated with `ffmpeg`.

Any scene whose render exits non-zero has its `stderr` and original JSON fed back to the LLM, which returns a corrected Manim class. This repeats up to three times; scenes that still fail are replaced with a placeholder frame.

5 EXPERIMENTS

5.1 BASELINE AND ABLATIONS

We evaluate a reference baseline, two primary retrieval variants of our proposed pipeline, and several ablations.

Baseline: Zero-shot generation (no context). We prompt a state-of-the-art LLM (GPT-5.4) with each benchmark question and no additional lecture context, relying solely on parametric knowledge.

Method (Global retrieval). The pipeline described in Section 4 with retrieval over the full course-wide chunk index.

Method (Lecture-filtered retrieval). The same pipeline restricted to chunks from the lecture containing the question.

Ablations. We additionally run ablations on several axes, including generation model (GPT-4o vs. GPT-5.4) and k for our top-k retrieval ($k = 8$ vs. $k = 3$).

5.2 EVALUATION SETUP

We report token-level F1 against the reference answer, exact match, recall, and an LLM-judged binary accuracy obtained by asking a separate judge model whether the predicted answer is consistent with the reference (judge prompt reproduced in Appendix A.2). We additionally report two retrieval diagnostics: *lecture match* is the fraction of questions for which any retrieved chunk comes from the correct lecture, and *temporal hit* is the fraction of questions for which any retrieved chunk additionally overlaps the annotator-specified time window. End-to-end latency is reported as wall-clock seconds across the full evaluation, including embedding the question, vector search, generation, and the offline judge call. All retrieval-augmented runs use the Azure OpenAI `text-embedding-3-large` encoder for indexing and queries; we vary the generator model (`gpt-4o`, `gpt-5.4`) and the retrieval depth $k \in \{3, 8\}$.

Per-course splits contain 52 (COGSCI C127), 50 (DATA 100), and 54 (CS 288) questions; a small number of API errors are tolerated by the harness (at most two on any COGSCI run, one on the DATA 100 no-context run, and one on each CS 288 RAG run), and these are excluded from per-question metric averages but counted toward wall-clock totals.

5.3 MAIN RESULTS

Table 2 reports the main results across all three courses. Lecture-filtered retrieval with `gpt-5.4` is the strongest setting on every course: best LLM-judged accuracy reaches 82.0% on DATA 100 (up from 20.4% no-context), 58.8% on COGSCI C127 (up from 32.7%), and 83.3% on CS 288 (up from 63.0%), while wall-clock evaluation stays in the same range as the no-context baseline. The retrieval quality shifts driven by lecture filtering are even larger: temporal hit rises from 0.558 (global) to 0.635 on COGSCI C127 and from 0.760 to 0.820 on DATA 100. On CS 288 the global filter already retrieves from the correct lecture in 100% of cases, so lecture filtering offers no additional retrieval benefit on that course but still improves answer quality (79.6% $\rightarrow$ 83.3% LLM-judged).

Two secondary findings are worth flagging. First, generator capacity matters more on lecture-grounded retrieved evidence than on no-context: switching from `gpt-4o` to `gpt-5.4` at the same $k = 8$ lifts DATA 100 LLM-judged accuracy from 66.0% to 82.0% while leaving the retrieval inputs unchanged, suggesting that the limiting factor on this course is the generator’s ability to reason over multimodal context rather than its world knowledge. Second, the optimal retrieval depth depends on the course: $k = 8$ is best on the retrieval-bound DATA 100 (78.0% $\rightarrow$ 82.0%), while $k = 3$ is best on the audio-heavy COGSCI C127 (58.8% at $k = 3$ vs. 55.8% at $k = 8$), where additional retrieved chunks tend to dilute rather than reinforce the relevant evidence. CS 288 ties between $k = 3$ and $k = 8$. We report both settings and treat $k = 8$ as a sensible default that never hurts substantially.

5.4 EXPLORATORY FULL-VIDEO CONDITIONING STRESS TEST

Beyond Table 2, we ran a separate evaluation in which only the questions answered *incorrectly* by the no-context baseline on COGSCI C127 (14 of 54) were re-presented to large multimodal models with the full lecture video uploaded as context. Table 3 reports the result. Despite having access to the full lecture, video-conditioned generation reached only 21.4% accuracy on this subset, and per-query latency rose to a mean of 195 seconds with a maximum exceeding eight minutes. The combination of long context windows and lecture-length inputs both degrades quality and pushes wall-clock time well past what is usable in an interactive student-facing setting. By contrast, our lecture-filtered RAG pipeline answers a comparable mix of questions in approximately three seconds.

Table 2: Main results. Lecture match and temporal hit measure retrieval quality; F1, EM, recall, and LLM match measure answer quality. Wall is total wall-clock seconds across the evaluation split. “Lect. filt.” denotes lecture-filtered retrieval (the *proposed* method) and “Global” denotes course-wide retrieval; both use the same chunk index. Bold marks the best LLM-judged accuracy per course; lecture-filtered retrieval with `gpt-5.4` is the strongest setting on every course, with the optimal k depending on whether the course is retrieval-bound or audio-heavy.

Course	Setting	Generator	k	Lect.	Temp.	F1	EM	Recall	LLM	Wall (s)
COGSCI C127	No context	<code>gpt-5.4</code>	—	—	—	0.207	0.077	0.226	0.327	121.9
	Lect. filt. (ours)	<code>gpt-4o</code>	8	1.000	0.635	0.289	0.135	0.307	0.442	206.9
	Lect. filt. (ours)	<code>gpt-5.4</code>	3	1.000	0.519	0.281	0.115	0.325	0.588	172.4
	Lect. filt. (ours)	<code>gpt-5.4</code>	8	1.000	0.635	0.316	0.154	0.346	0.558	168.8
	Global	<code>gpt-5.4</code>	8	0.635	0.558	0.370	0.154	0.405	0.538	165.5
DATA 100	No context	<code>gpt-5.4</code>	—	—	—	0.179	0.120	0.183	0.204	122.8
	Lect. filt. (ours)	<code>gpt-4o</code>	8	1.000	0.820	0.606	0.520	0.650	0.660	182.3
	Lect. filt. (ours)	<code>gpt-5.4</code>	3	1.000	0.740	0.711	0.620	0.742	0.780	157.2
	Lect. filt. (ours)	<code>gpt-5.4</code>	8	1.000	0.820	0.756	0.680	0.790	0.820	163.5
	Global	<code>gpt-5.4</code>	8	0.940	0.760	0.726	0.660	0.750	0.780	159.8
CS 288	No context	<code>gpt-5.4</code>	—	—	—	0.492	0.278	0.481	0.630	113.9
	Lect. filt. (ours)	<code>gpt-4o</code>	8	1.000	0.944	0.585	0.259	0.595	0.796	217.5
	Lect. filt. (ours)	<code>gpt-5.4</code>	3	1.000	0.944	0.622	0.315	0.647	0.833	166.1
	Lect. filt. (ours)	<code>gpt-5.4</code>	8	1.000	0.944	0.618	0.296	0.644	0.833	183.0
	Global	<code>gpt-5.4</code>	8	1.000	0.944	0.601	0.278	0.631	0.796	180.6

Table 3: Video-conditioned generation (Baseline 2) on the subset of 14 COGSCI C127 questions that the no-context baseline answered incorrectly. Aggregate accuracy and F1 are reported across the subset; latency statistics describe per-question wall-clock time in seconds.

Metric	Aggregate	Per-question latency (s)
Accuracy (%)	21.4%	—
F1	0.23	—
Latency (s)	—	mean 195.4, median 152.6, min 7.9, max 508.2

6 ANALYSIS

6.1 LECTURE FILTERING HELPS WHERE EVIDENCE IS LOCAL

Lecture-filtered retrieval is uniformly competitive with or better than global retrieval on every course in Table 2, but the size of the gap depends on how localized the evidence is. On DATA 100, where questions reference specific demos, dataframes, and slides, the in-lecture witness is the only one that contains the answer, and lecture filtering improves LLM-judged accuracy by 4 absolute points over the same generator at the same k . On CS 288, where the global filter already retrieves from the correct lecture in 100% of cases, lecture filtering nonetheless improves answer quality by a similar margin (83.3% vs. 79.6% LLM-judged), suggesting a benefit even from breaking ties between similar-sounding chunks across nearby lectures. On COGSCI C127, which is dominated by audio-only questions whose phrasing recurs across lectures, the gap between filtering modes is smallest.

6.2 LOCAL RETRIEVAL BACKEND FOLLOW-UP

We additionally evaluated a local retrieval backend on a 48-question subset of COGSCI C127. This run used timestamped lecture chunks from `lecture_chunks_30_6.jsonl`, local embeddings from BAAI/bge-small-en-v1.5, and retrieved the top $k = 8$ chunks per question. Unlike the Azure-indexed results in Table 2, this setting uses a local vector store and a slightly different evaluation subset, so we treat it as a backend follow-up rather than a directly controlled replacement for the main COGSCI C127 row.

Setting	N	Temp. Hit	Any Overlap	F1	EM	LLM Match	Time / Q
Local RAG, BGE-small, $k=8$	48	89.6	93.8	41.3	14.6	85.4	6.76s

Table 4: Local-backend COGSCI C127 lecture QA follow-up. Temporal Hit requires a retrieved chunk to match the question lecture number and overlap the gold timestamp interval. Any Overlap measures whether any retrieved chunk overlaps the gold timestamp interval. F1, exact match (EM), and LLM Match are reported as percentages. Time per question includes retrieval and generation.

The local backend achieved strong retrieval grounding: 89.6% temporal hit rate and 93.8% timestamp overlap rate. This indicates that the retriever usually found lecture chunks overlapping the annotated gold interval. Answer quality was also substantially higher under the LLM-judge metric, reaching 85.4% LLM Match. Lexical metrics remained more modest, with 41.3% F1, 14.6% exact match, and 46.7% recall. This gap reinforces a pattern observed elsewhere in our experiments: exact match and token-level F1 can understate answer correctness in lecture QA, where correct answers are often short paraphrases of spoken or visual lecture content rather than exact string matches.

Latency was dominated by generation rather than retrieval. Average retrieval time was only 0.056 seconds per question, while average generation time was 6.70 seconds. Thus, the local backend itself added essentially negligible overhead, but the end-to-end per-question time was approximately 6.76 seconds once generation was included. The full 48-question run completed in 420.2 seconds.

These results suggest that the chunk representation and retrieval formulation are not tied to the Azure search backend. A lightweight local retriever with BGE-small embeddings can recover temporally relevant evidence at high rates and can support strong answer accuracy when paired with a capable generator. However, because this run uses a different question count and backend configuration than the main COGSCI C127 rows in Table 2, we avoid interpreting the 85.4% LLM Match as a strict apples-to-apples improvement without a controlled rerun on the same subset.

6.3 LATENCY.

End-to-end wall-clock time on the full evaluation harness ranges from 113.9 s (no context, CS 288) to 217.5 s (lecture-filtered `gpt-4o`, $k = 8$, CS 288), corresponding to roughly 2–4 seconds per question. Decomposing the per-question budget for our reference configuration (lecture-filtered `gpt-5.4`, $k = 8$): retrieval averages 1.01 s on DATA 100, 1.06 s on COGSCI C127, and 1.07 s on CS 288, while generation averages 1.26 s, 1.18 s, and 1.31 s respectively. The same pipeline with `gpt-4o` as the generator is uniformly slower at the generation step (1.43 s on DATA 100, 1.60 s on COGSCI C127, 1.73 s on CS 288), which accounts for most of the wall-clock gap between the two generator settings in Table 2. The offline LLM judge adds a further 1.0–1.3 s per question. All of these numbers compare favorably to the 195-second *per-question* mean for video-conditioned generation in Table 3 and remain within the budget for interactive student-facing use.

7 LIMITATIONS AND BROADER IMPACT

Our benchmark covers three courses at one institution and 156 questions; absolute numbers should not be interpreted as a population-level estimate of lecture QA difficulty. The annotation process is single-pass, and although we report category counts we do not measure inter-annotator agreement; verifying the dataset with course staff is a planned next step. The pipeline relies on commercial LLM and VLM APIs whose behavior may drift over time, which limits exact reproducibility; we mitigate this by releasing all prompts, retrieval indices, and per-question raw outputs alongside the metric scripts. Finally, deploying lecture QA tools in real classrooms raises questions about academic integrity (whether students should ask the system questions during exams) and about consent for instructors whose recorded lectures are indexed; we recommend that any deployment require explicit instructor opt-in and clearly distinguish review-mode use from assessment-mode use.

8 CONCLUSION

We presented a visual–temporal RAG pipeline for lecture question answering, together with a manually annotated benchmark of 156 questions across three university courses. Time-aligned multimodal chunking with visual carry-forward bundles OCR, ASR, and VLM frame descriptions inside a single retrieval unit and lifts LLM-judged accuracy from 20.4% to 82.0% on DATA 100, 32.7% to 58.8% on COGSCI C127, and 63.0% to 83.3% on CS 288, while keeping per-question latency on the order of three seconds. The remaining failure modes are fine-grained chart reading and motion-dependent reasoning, both of which point toward a final-stage frame-image pass that we leave for future work.

AUTHOR CONTRIBUTIONS

All three authors contributed jointly to dataset construction, pipeline implementation, and writing. Meenakshi Mittal and Micah Mok led evaluation and retrieval, and Anthony Kuang led output generation.

GENAI ACKNOWLEDGEMENT SECTION

We acknowledge the use of Large Language Models and Code Completion Tools to assist in the development of the pipeline. We used the outputs to understand error messages, look up API methods, and tab-complete small snippets of functions with pseudocode. We have the ability to explain and even independently replicate the work done in this document if asked by an instructor.

REFERENCES

- Dong Won Lee, Chaitanya Ahuja, Paul Pu Liang, Sanika Natu, and Louis-Philippe Morency. Multimodal lecture presentations dataset: Understanding multimodality in educational slides. *arXiv preprint arXiv:2208.08080*, 2022.
- Chirantan Mahipal, Lawrence Angrave, Yuren Xie, Biswadeep Chatterjee, Hongyu Wang, and Zhengru Qian. “What did I just miss?!” Presenting ClassTranscribe, an automated live-captioning and text-searchable lecture video system, and related pedagogical best-practices. In *ASEE Annual Conference and Exposition, Conference Proceedings*, 2019.
- Sourjyadip Ray, Shubham Sharma, Somak Aditya, and Pawan Goyal. EduVidQA: Generating and evaluating long-form answers to student questions based on lecture videos. *arXiv preprint arXiv:2509.24120*, 2025.
- Xubin Ren, Lingrui Xu, Long Xia, Shuaiqiang Wang, Dawei Yin, and Chao Huang. VideoRAG: Retrieval-augmented generation with extreme long-context videos. *arXiv preprint arXiv:2502.01549*, 2025.
- Nina Shvetsova, Anna Kukleva, Xudong Hong, Christian Rupprecht, Bernt Schiele, and Hilde Kuehne. HowToCaption: Prompting LLMs to transform video annotations at scale. *arXiv preprint arXiv:2310.04900*, 2023.
- Enxin Song, Wenhao Chai, Weili Xu, Jianwen Xie, Yuxuan Liu, and Gaoang Wang. Video-MMLU: A massive multi-discipline lecture understanding benchmark. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, pp. 6158–6172, 2025.
- Himanshu Wadhwa, T. Karthikeyan, Mausam, and Manish Gupta. Towards multimodal question answering in educational domain. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pp. 1638–1649, Mumbai, India, December 2025.
- Minghan Wang, Yuxia Wang, Thuy-Trang Vu, Ehsan Shareghi, and Gholamreza Haffari. Exploring the potential of multimodal LLM with knowledge-intensive multimodal ASR. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 13274–13288, 2024.
- Zhuang Yu, Lei Shen, Jing Zhao, and Shiliang Sun. LEMON: How well do MLLMs perform temporal multimodal understanding on instructional videos? *arXiv preprint arXiv:2601.20705*, 2026.

A APPENDIX

A.1 GENERATOR SYSTEM PROMPT

The same system prompt is used by the generator across all retrieval-augmented and no-context conditions. The retrieved context, when present, is supplied in the user message rather than the system message:

```
You are a teaching assistant for a lecture course.
Use the provided lecture context to answer the question.
Answer like an exam answer key: the shortest correct
phrase, name, label, number, or sentence.
Reply as briefly and concisely as possible. Respond with
only the answer. No explanation, preamble, or extra detail.
If the question is to describe the content of a video,
rather than provide the title of the video, try to summarize
in fifty words the content of that video from the provided
lecture context.
If you don't know, just say so.
```

A.2 LLM JUDGE PROMPT

The judge is the same Azure OpenAI deployment used as the generator (gpt-5.4). Its system message is ``You are a strict answer equivalence judge.`` and its user message is templated as follows; the parser accepts case-insensitive prefixes true or false and otherwise records a parse error:

```
Decide whether the system answer matches the ground truth
answer for the question.
Return only true or false.
Question: {question}
Ground truth answer: {gold}
System answer: {response}
```

A.3 INDEXING AND RETRIEVAL DETAILS

Each lecture is processed offline once. OCR is run on every sampled frame whose visual content has changed since the previous sample, with chunks dropping OCR lines under twelve characters to suppress menu chrome. ASR is run end-to-end on the audio track and post-aligned to chunk boundaries by start timestamp. VLM frame descriptions are generated on the same change-detected frames as OCR. Within each chunk, OCR and frame rows are subsampled to a minimum gap of ten seconds to avoid embedding near-duplicate visual descriptions, and the latest OCR row and latest frame-description row strictly preceding the window start are carried forward.

The Azure AI Search index uses an HNSW vector profile over a single `content_vector` field, embedded with Azure OpenAI `text-embedding-3-large`. Filterable fields (`course_id`, `lecture_id`, `lecture_number`, `sources`, `carried_forward`) support the global and lecture-filtered retrieval scopes evaluated in Section 5. Document IDs are generated by base64-url-encoding the chunk identifier so that subsequent ingestion runs can perform idempotent `mergeOrUpload` operations.